

Отлично, давайте разберем эту лабораторную работу шаг за шагом, но не в коде, а вручную — так, как вы должны будете делать на бумаге или в уме, чтобы понять суть каждого пункта.

У вас есть **логическая функция** (например, $!(a \rightarrow !b) \vee c$) с переменными (a, b, c, d, e — до 5 штук).

Все задания будут выполняться вручную по следующему алгоритму.

1. Построение таблицы истинности

Что делать:

- Определите, сколько переменных в функции (например, a, b, c — 3 переменные $\rightarrow 2^3 = 8$ строк).
- Перебираете все возможные комбинации переменных (000, 001, 010, ..., 111).
- Для каждой комбинации вычисляете значение функции, подставляя значения в выражение.

Пример:

Функция: $f(a, b, c) = (a \vee b) \wedge \neg c$

Таблица истинности:

a	b	c	f
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1

1	1	1	0
---	---	---	---

Вы просто подставляете значения и считаете.

2. Построение СДНФ и СКНФ по таблице истинности

СДНФ (совершенная дизъюнктивная нормальная форма)

- Смотрите строки, где $f = 1$.
- Для каждой такой строки составляете конъюнкцию (И) переменных: если переменная = 0, берем $\neg x$, если 1 — берем x .
- Все эти конъюнкции соединяете через $\vee$ (ИЛИ).

Пример: из таблицы выше $f=1$ в строках:

$010 \rightarrow \neg a \wedge b \wedge \neg c$

$100 \rightarrow a \wedge \neg b \wedge \neg c$

$110 \rightarrow a \wedge b \wedge \neg c$

СДНФ: $(\neg a \wedge b \wedge \neg c) \vee (a \wedge \neg b \wedge \neg c) \vee (a \wedge b \wedge \neg c)$

СКНФ (совершенная конъюнктивная нормальная форма)

- Смотрите строки, где $f = 0$.
- Для каждой такой строки составляете дизъюнкцию (ИЛИ) переменных: если переменная = 1, берем $\neg x$, если 0 — берем x .
- Все эти дизъюнкции соединяете через $\wedge$ (И).

Пример: $f=0$ в строках 000, 001, 011, 101, 111:

$000 \rightarrow a \vee b \vee c$

$001 \rightarrow a \vee b \vee \neg c$

$011 \rightarrow a \vee \neg b \vee \neg c$

$101 \rightarrow \neg a \vee b \vee \neg c$

$111 \rightarrow \neg a \vee \neg b \vee \neg c$

СКНФ: $(a \vee b \vee c) \wedge (a \vee b \vee \neg c) \wedge (a \vee \neg b \vee \neg c) \wedge (\neg a \vee b \vee \neg c) \wedge (\neg a \vee \neg b \vee \neg c)$

3. Числовая форма для СДНФ и СКНФ

СДНФ: выписываете номера наборов, где $f=1$, в виде чисел.

Номер набора — это двоичное число переменных, переведенное в десятичное.

Пример: набор 010 ($a=0, b=1, c=0$) $\rightarrow$ 2 (если порядок abc).

В СДНФ из примера наборы: 010(2), 100(4), 110(6) $\rightarrow$ числовая форма: $\vee(2, 4, 6)$

СКНФ: выписываете номера наборов, где $f=0$, но в другой записи:

Обычно пишут $\wedge(0, 1, 3, 5, 7)$ для СКНФ.

4. Индексная форма функции

- Строите строку значений функции от набора 0 до набора $2^n - 1$.

Пример из таблицы: $f = 0, 0, 1, 0, 1, 0, 1, 0$

- Это двоичное число: 00101010 (читается от набора 0 до набора 7).
- Переводите в десятичное:

$$0 \cdot 128 + 0 \cdot 64 + 1 \cdot 32 + 0 \cdot 16 + 1 \cdot 8 + 0 \cdot 4 + 1 \cdot 2 + 0 \cdot 1 = 32 + 8 + 2 = 42$$

Индекс функции = 42.

5. Классы Поста (определение принадлежности)

Классы Поста — это 5 свойств функции. Нужно проверить, входит ли функция в каждый класс.

1. **T0 (сохраняет 0)** — $f(0, 0, \dots, 0) = 0$.

Если да $\rightarrow$ входит.

2. **T1 (сохраняет 1)** — $f(1, 1, \dots, 1) = 1$.

3. **S (самодвойственная)** — $f(\neg x_1, \neg x_2, \dots) = \neg f(x_1, x_2, \dots)$ для всех наборов.

4. **M (монотонная)** — при любом увеличении набора ($0 \rightarrow 1$) значение не уменьшается.

5. **L (линейная)** — полином Жегалкина содержит только конъюнкции не более чем одной переменной (без произведений переменных).

Проверяете вручную по таблице истинности.

6. Полином Жегалкина

Метод:

- Используете таблицу истинности.
- Строите полином вида: $a \oplus b \oplus c \oplus ab \oplus ac \oplus bc \oplus abc \oplus \dots$ ($\oplus = \text{XOR}$).
- Метод неопределенных коэффициентов или метод треугольника.

Пример для 2 переменных:

$f(0,0)=0, f(0,1)=1, f(1,0)=1, f(1,1)=0 \rightarrow$

Полином: $f = a \oplus b$

Вручную: выписываете все возможные конъюнкции переменных и решаете систему.

7. Поиск фиктивных переменных

Что делать:

- Переменная фиктивная, если значение функции не зависит от нее.
- Проверяете: для всех наборов, отличающихся только этой переменной, f одинакова.

Пример: $f(a,b) = a \vee \neg a \rightarrow$ всегда $1 \rightarrow a$ и b фиктивны.

Вручную: фиксируете все остальные переменные, меняете эту — если f не меняется $\rightarrow$ фиктивная.

8. Булева дифференциация

Частная производная по x :

$$\partial f / \partial x = f(x=1) \oplus f(x=0) \text{ (XOR)}$$

То есть берете таблицу, фиксируете все переменные, кроме x , смотрите, меняется ли f при переключении x . Если да $\rightarrow$ производная=1, иначе 0.

Смешанная производная: $\partial^2 f / \partial x \partial y$ = сначала дифференцируете по x , потом по y (или наоборот).

Пример: $f = x \wedge y$

$$\partial f / \partial x = (1 \wedge y) \oplus (0 \wedge y) = y \oplus 0 = y$$

9. Минимизация расчетным методом (склеивание)

Алгоритм:

1. Записать СДНФ.
2. Склеивать конституенты, отличающиеся только одной переменной:
 $(x \wedge y \wedge z) \vee (\neg x \wedge y \wedge z) = (y \wedge z)$
3. Повторять, пока можно.
4. Проверить, нет ли лишних импликант (те, которые покрываются другими).

Пример в методичке:

$$(\neg abc) \vee (a\neg b\neg c) \vee (a\neg bc) \vee (ab\neg c) \vee (abc) \rightarrow \text{после склеивания:}$$

$$a \vee bc$$

10. Расчетно-табличный метод

1. Делаете склеивание как в п.9 $\rightarrow$ получаете импликанты.
2. Строите таблицу: строки — импликанты, столбцы — исходные конституенты (где $f=1$).
3. Ставите крестики, если импликанта покрывает конституенту.

4. Выбираете минимальное покрытие (обычно сначала берете строки с уникальными столбцами).

Пример в методичке:

Импликаны a , ac , $bc \rightarrow$ таблица показывает, что ac лишняя, остается $a \vee bc$.

11. Минимизация картой Карно (табличный метод)

Для 3-4 переменных:

1. Рисуете таблицу Карно (например, 2×4 для 3 переменных).
2. Заполняете 1 в клетках, где $f=1$.
3. Группируете 1 в прямоугольники размера 1, 2, 4, 8... (степени 2), максимального размера.
4. Каждая группа — одна импликанта (общие переменные выносятся).
5. Соединяете импликаны через $\vee$.

Пример из методички:

Карта для 3 переменных:

$a \backslash bc$: 00 01 11 10

0: 0 0 1 0

1: 1 1 1 1

Группы:

- Вся строка $a=1 \rightarrow$ импликанта a
- Столбец $bc=11 \rightarrow$ импликанта bc

Результат: $a \vee bc$

Знак $\oplus$ означает **сложение по модулю 2** (также называется **XOR** — исключающее ИЛИ).

Правила работы:

A	B	$A \oplus B$
---	---	--------------

0	0	0
0	1	1
1	0	1
1	1	0

Словесно: результат равен 1, когда аргументы **различны**, и 0, когда **одинаковы**.

Свойства:

- $A \oplus 0 = A$ (ноль — нейтральный элемент)
- $A \oplus 1 = \neg A$ (инверсия)
- $A \oplus A = 0$
- $A \oplus B = B \oplus A$ (коммутативность)
- $(A \oplus B) \oplus C = A \oplus (B \oplus C)$ (ассоциативность)
- $A \oplus B = (A \wedge \neg B) \vee (\neg A \wedge B)$ (выражение через И, ИЛИ, НЕ)

В контексте полинома Жегалкина:

Запись $f = a_0 \oplus a_1 \cdot x_1 \oplus a_2 \cdot x_2$ означает, что функция вычисляется как **XOR** (сложение по модулю 2) этих слагаемых.

Пример: $1 \oplus x \oplus y$ — это таблица истинности:

x	y	$1 \oplus x \oplus y$
0	0	$1 \oplus 0 \oplus 0 = 1$
0	1	$1 \oplus 0 \oplus 1 = 0$
1	0	$1 \oplus 1 \oplus 0 = 0$
1	1	$1 \oplus 1 \oplus 1 = 1$

Это функция **эквивалентности** ($x \equiv y$).

Ниже приведён пошаговый алгоритм, как **вручную** проверить, принадлежит ли булева функция (заданная таблицей истинности) каждому из **5 классов Поста**.

1. T0 (сохраняет 0)

Проверка:

Посмотри на **первую строку** таблицы истинности (все аргументы = 0).

Если значение функции в этой строке **0** → функция входит в T0.

Если **1** → не входит.

Пример (для 2 переменных):

x	y	f
0	0	0 ← смотрим сюда
0	1	1
1	0	1
1	1	0

→ $f(0,0)=0 \rightarrow \in T0$

2. T1 (сохраняет 1)

Проверка:

Посмотри на **последнюю строку** таблицы (все аргументы = 1).

Если $f(1,1,\dots,1) = 1 \rightarrow \in T1$.

Если = **0** → не входит.

Для того же примера:

x	y	f
0	0	0
...	...	...

1	1	0 ← последняя строка
---	---	----------------------

→ $f(1,1)=0 \rightarrow \notin T1$

3. S (самодвойственная)

Проверка вручную:

Разбей все строки таблицы на **пары** (набор, отрицание набора).

Для каждого набора проверь:

$$f(\neg x_1, \neg x_2, \dots) = \neg f(x_1, x_2, \dots)$$

Пример для 2 переменных (наборы: 00, 01, 10, 11):

x	y	f
0	0	a
0	1	b
1	0	c
1	1	d

Пары:

- (00 и 11): $f(00)=a, f(11)=d \rightarrow$ требуется **d = $\neg a$**
- (01 и 10): $f(01)=b, f(10)=c \rightarrow$ требуется **c = $\neg b$**

Важно:

- Если число строк нечётное (для чётного n) → самодвойственной **не бывает** (кроме n=0).
- Для самодвойственной функции нужно, чтобы на противоположных наборах значения были **противоположны**.

4. М (монотонная)

Проверка:

Сравнивай все пары наборов, где один набор **доминирует** над другим (везде $0 \rightarrow 1$, но не обязательно строго).

Если для любой такой пары:

если набор $A \leq$ набор B (покомпонентно), то $f(A) \leq f(B)$ — функция монотонна.

Пример нарушения:

Набор $A = (0, 1)$, $B = (1, 1)$

$f(A)=1$, $f(B)=0 \rightarrow$ нарушение ($1 > 0$, хотя $A < B$).

Как проще:

Иди по таблице, проверяй, что при замене какого-то 0 на 1 (в любом разряде) значение функции **не становится 0, если было 1**.

Можно использовать **соседние по Хэммингу** наборы (отличаются в 1 бите).

Если $f(\dots, 0, \dots)=1$, а $f(\dots, 1, \dots)=0 \rightarrow$ нарушение монотонности.

5. L (линейная)

Проверка:

Нужно построить **полином Жегалкина**.

Алгоритм (метод неопределённых коэффициентов):

1. Запиши функцию как сумму по модулю 2:
$$f(x_1 \dots x_n) = c_0 \oplus c_1 \cdot x_1 \oplus c_2 \cdot x_2 \oplus \dots \oplus c_n \cdot x_n \oplus c_{12} \cdot x_1 x_2 \oplus \dots$$
2. Подставляй по очереди **все наборы** (удобно в порядке возрастания двоичных чисел).
3. На каждом шаге вычисляй новый коэффициент как значение функции на текущем наборе **минус** (xor) уже найденные слагаемые.

Более простой способ:

- Если в **СДНФ** (дизъюнкция конъюнкций) все конъюнкции содержат **не более одной переменной** (т.е. это хог переменных с константой) → линейна.
- Проще: построй таблицу истинности и проверь, можно ли представить f как: $f = a_0 \oplus a_1 \cdot x_1 \oplus a_2 \cdot x_2 \oplus \dots \oplus a_n \cdot x_n$ (без произведений типа $x_1 x_2$).

Пример линейной:

$$x \oplus y \oplus 1$$

Нелинейной:

xy (конъюнкция) – потому что есть произведение $x \cdot y$.

Итог: как быстро проверить все 5 классов

1. **T0** → первая строка = 0?
2. **T1** → последняя строка = 1?
3. **S** → противоположные наборы дают противоположные значения.
4. **M** → нигде 1 не идёт перед 0 при увеличении набора.
5. **L** → можно ли записать как сумму (хор) переменных и константы?

Если нужно, могу показать **пример проверки** для конкретной таблицы истинности (укажите число переменных и столбец f).

Отличный вопрос! Вы привели классическую задачу **минимизации булевых функций методом склеивания** (также называется методом Квайна или методом попарного склеивания).

Разберем ваш пример пошагово, как из набора из 5 термов получилось $a \vee b \vee c$.

Исходные данные (СДНФ)

Обозначения: $\neg a$ — это a с чертой или отрицание. В методичке, видимо, a без черты — это истина, а $\neg a$ — ложь. Ваш пример:

$$(\neg a \vee b \vee c) \vee (a \vee \neg b \vee \neg c) \vee (a \vee \neg b \vee c) \vee (a \vee b \vee \neg c) \vee (a \vee b \vee c)$$

Давайте перепишем в более удобном для глаза виде, используя точки для И (конъюнкции) и опуская скобки там, где это понятно:

$$\neg a \cdot b \cdot c \vee a \cdot \neg b \cdot \neg c \vee a \cdot \neg b \cdot c \vee a \cdot b \cdot \neg c \vee a \cdot b \cdot c$$

Шаг 1. Расставляем переменные по порядку

Удобно, чтобы все термы имели одинаковый порядок переменных. Пусть порядок будет **(a, b, c)**. Тогда каждый терм — это тройка букв.

- $\neg a \ b \ c \rightarrow (0, 1, 1)$
- $a \ \neg b \ \neg c \rightarrow (1, 0, 0)$
- $a \ \neg b \ c \rightarrow (1, 0, 1)$
- $a \ b \ \neg c \rightarrow (1, 1, 0)$
- $a \ b \ c \rightarrow (1, 1, 1)$

Шаг 2. Первое склеивание

Ищем пары термов, которые отличаются **ровно в одной позиции** (по одной переменной), а в остальных совпадают.

Склеивание по a (меняется a)

Ищем пары, у которых b и c одинаковы.

- **b=1, c=1:**
 - $\neg a \ b \ c \ (0, 1, 1)$ и $a \ b \ c \ (1, 1, 1) \rightarrow$ отличаются по **a**.

$$\text{Склеиваем: } (\neg a \ b \ c) \vee (a \ b \ c) = (b \ c)$$

Правило: если есть X и $\neg X$ в одной позиции, а остальные совпадают, то результат — конъюнкция общих переменных.

Здесь общие: b и c.

Получили **b c** (то есть $b \wedge c$).

Других пар с одинаковыми b и c нет (у нас только $\neg a \ b \ c$ и $a \ b \ c$).

Склеивание по b (меняется b)

Ищем пары с одинаковыми a и c.

- **a=1, c=0:**

- $a \neg b \neg c (1,0,0)$ и $a b \neg c (1,1,0) \rightarrow$ отличаются по **b**.

Склеиваем: общие a и $\neg c \rightarrow a \neg c$

- **a=1, c=1:**

- $a \neg b c (1,0,1)$ и $a b c (1,1,1) \rightarrow$ отличаются по **b**.

Склеиваем: общие a и c $\rightarrow a c$

Склеивание по c (меняется c)

Ищем пары с одинаковыми a и b.

- **a=1, b=0:**

- $a \neg b \neg c (1,0,0)$ и $a \neg b c (1,0,1) \rightarrow$ отличаются по **c**.

Склеиваем: общие a и $\neg b \rightarrow a \neg b$

- **a=1, b=1:**

- $a b \neg c (1,1,0)$ и $a b c (1,1,1) \rightarrow$ отличаются по **c**.

Склеиваем: общие a и b $\rightarrow a b$

Результаты первого склеивания (простые импликанты):

- $b c$ (из склеивания по a)
- $a \neg c$
- $a c$
- $a \neg b$
- $a b$

То есть получили набор:

$bc \vee a\neg c \vee ac \vee a\neg b \vee ab$

Но это еще не финал — некоторые из этих термов могут склеиваться дальше!

Шаг 3. Второе склеивание

Теперь работаем с новыми термами. Они имеют длину 2 (по две переменные).
Опять ищем пары, отличающиеся в одной переменной.

Рассмотрим термы:

1. bc (переменные b, c; a отсутствует)

2. $a \neg c$
3. ac
4. $a \neg b$
5. ab

Склеивание возможно, если у двух термов одинаковый набор переменных **кроме одной**, и в этой одной одна из них имеет переменную, а вторая — ее отрицание или наоборот.

Важное замечание: bc не содержит a , поэтому при склеивании с любым термом, содержащим a , они отличаются более чем в одной переменной, так как bc "молчит" про a . Это не подходит.

Смотрим на пары среди тех, что содержат a :

- $a \neg c$ и $ac \rightarrow$ отличаются по c .

Общее: a .

Склеиваем: $a \neg c \vee ac = a$

- $a \neg b$ и $ab \rightarrow$ отличаются по b .

Общее: a .

Склеиваем: $a \neg b \vee ab = a$

- $a \neg c$ и $a \neg b \rightarrow$ отличаются и по b , и по c ? Не подходит, разница в двух переменных.
- ac и $ab \rightarrow$ отличаются по b и c ? Не подходит.
- $a \neg b$ и $a \neg c \rightarrow$ отличаются по b и c ? Не подходит.
- $a \neg b$ и $ac \rightarrow$ различаются по b и c ? Не подходит.
- bc ни с кем не склеивается, кроме гипотетического $\neg b \vee c$ или $b \rightarrow c$, но таких у нас нет.

Итак, на втором шаге склеивания получили **a** (из склейки $a \neg c$ и ac , а также из $a \neg b$ и ab).

Результаты второго склеивания: a

Шаг 4. Что осталось?

Первый шаг дал: bc , $a \neg c$, ac , $a \neg b$, ab

Второй шаг дал: a (поглощает $a\bar{c}$, ac , $a\bar{b}$, ab — все, где есть a и что-то еще).

Остаются:

- a — уже есть как простой терм.
- bc — не склеился ни с чем.

Больше склеивать нечего.

Шаг 5. Проверка на лишние импликанты (поглощение)

Теперь имеем множество простых импликант: $\{ a, bc \}$.

Проверяем, не покрывает ли одна импликанта другую:

- a покрывает все исходные термы, где $a=1$ (все, кроме $\bar{a}bc$).
- bc покрывает $\bar{a}bc$ и abc . Но abc уже покрыт a .

Однако bc нужен для покрытия $\bar{a}bc$ (потому что в $\bar{a}bc$ $a=0$, и a его не покрывает).

Оба нужны.

Итог: $a \vee bc$

Альтернативный подход — карты Карно

Для 3 переменных проще всего использовать карту Карно:

	bc			
	00	01	11	10
a=0	0	0	1	0
a=1	1	1	1	1

Расставляем единицы по вашим термам:

- $\bar{a}bc$ (0,1,1) → ячейка $a=0$, $bc=11$ → единица
- $a\bar{b}\bar{c}$ (1,0,0) → $a=1$, $bc=00$ → единица
- $a\bar{b}c$ (1,0,1) → $a=1$, $bc=01$ → единица
- $ab\bar{c}$ (1,1,0) → $a=1$, $bc=10$ → единица
- abc (1,1,1) → $a=1$, $bc=11$ → единица

Группируем:

- Все четыре единицы в $a=1$ (верхняя строка $a=1$) — это **a**.
- Осталась одна единица в $a=0$, $bc=11$ — это **b c**.

Итог: $a \vee bc$

Короткий ответ для вашего примера:

Последовательное склеивание пар, отличающихся одной переменной, приводит к двум простым импликантам: a (покрывает все случаи с $a=1$) и bc (покрывает случай $\neg a \wedge b \wedge c$). Их дизъюнкция и есть минимальная форма.

Отличный запрос! Вы описали **расчетно-табличный метод** (также известен как **метод Квайна-Мак-Класки** или просто метод импликантных таблиц). Это формализация того, как после получения всех простых импликант (методом склеивания из п.9) выбрать **минимальное покрытие** исходных конституент (минтермов).

Разберем **ваш пример** из методички.

Дано (после склеивания п.9)

Предположим, исходная функция имела минтермы, которые в итоге покрываются тремя простыми импликантами:

- **a**
- **ac**
- **bc**

И утверждается, что **ac** — лишняя, а минимальная форма — $a \vee bc$.

Шаг 1. Восстановим исходные конституенты (минтермы)

Чтобы понять, почему ac лишняя, нужно знать, какие минтермы были в исходной СДНФ. Поскольку в методичке это не приведено, **реконструируем их** из импликант, предполагая, что других минтермов не было (все минтермы покрываются хотя бы одной из трех импликант).

Возможные минтермы для трех переменных (a, b, c):

- Покрываются **a**: все минтермы, где $a=1$:

$a b c (111), a b \neg c (110), a \neg b c (101), a \neg b \neg c (100)$

- Покрываются **ac**: все минтермы, где $a=1$ и $c=1$:

$a b c (111), a \neg b c (101)$

- Покрываются **bc**: все минтермы, где $b=1$ и $c=1$:

$a b c (111), \neg a b c (011)$

Объединение всех покрываемых минтермов (по трем импликантам):

- От a : 100, 101, 110, 111
- От ac : 101, 111 (уже есть)
- От bc : 011, 111

Итоговый набор исходных конституент (где $f=1$):

011, 100, 101, 110, 111

То есть исходная функция имела СДНФ:

$\neg a b c \vee a \neg b \neg c \vee a \neg b c \vee a b \neg c \vee a b c$

(это тот же пример, что и в предыдущем вопросе, но в методичке он приведен как пример для табличного метода).

Шаг 2. Строим таблицу покрытия

Строки — импликанты (простые): **a, ac, bc**.

Столбцы — минтермы исходной функции (где $f=1$):

(обозначим их для краткости как номера двоичных наборов:

011, 100, 101, 110, 111)

Ставим **крестик** (или 1), если импликанта **покрывает** минтерм (т.е. минтерм входит в импликанту).

- **a** покрывает все минтермы с $a=1$: 100, 101, 110, 111

(НЕ покрывает 011, т.к. там $a=0$)

- **ac** покрывает минтермы с $a=1$ и $c=1$: 101, 111
- **bc** покрывает минтермы с $b=1$ и $c=1$: 011, 111

Таблица:

Импликанта	011	100	101	110	111
a		x	x	x	x
ac			x		x
bc	x				x

Шаг 3. Находим ядро покрытия (обязательные импликанты)

Ядро — это импликанты, которые **единственные** покрывают какой-либо столбец (минтерм). Если столбец имеет только один крестик, эта импликанта обязательна.

Смотрим по столбцам:

- **011**: крестик только у **bc**. → **bc** обязательна.
- **100**: крестик только у **a**. → **a** обязательна.
- **101**: крестики у **a** и **ac** → не единственная.
- **110**: крестик только у **a**. → **a** (уже в ядре).
- **111**: крестики у **a**, **ac**, **bc** → не единственная.

Ядро: **a**, **bc**.

Шаг 4. Проверяем, покрывают ли импликанты ядра все столбцы

Покрытие **a** ∪ **bc**:

- **a** покрывает: 100, 101, 110, 111
- **bc** покрывает: 011, 111

Вместе покрывают **все** столбцы: 011, 100, 101, 110, 111.

Импликанта **ac** не нужна — она не покрывает ни одного столбца, который не был бы уже покрыт.

Шаг 5. Итог

Минимальная форма: **$a \vee bc$** .

Импликанта **ac** — **лишняя** (избыточная), так как все минтермы, которые она покрывает (101 и 111), уже покрываются ядерными импликантами.

Общий алгоритм (коротко)

1. **Построить таблицу:** строки — простые импликанты, столбцы — минтермы.
2. **Отметить покрытия** крестиками.
3. **Найти столбцы с единственным крестиком** → включить соответствующую импликанту в ядро.
4. **Удалить** покрытые ядром столбцы (и строки, если они целиком покрыты — но обычно строки не удаляют, а проверяют, остались ли непокрытые столбцы).
5. **Если остались непокрытые столбцы**, выбрать минимальное дополнительное покрытие (обычно из строк, покрывающих максимальное число оставшихся столбцов — это задача о покрытии, но для малых таблиц решается перебором).
6. **Записать дизъюнкцию** ядерных и выбранных дополнительных импликант.

В вашем примере шаги 4-5 не потребовались, так как ядро уже покрыло всё.

Почему в методичке сказано про « ac лишняя»?

Импликанта **ac** является **избыточной**, потому что:

- Она не входит в ядро (нет уникального столбца только для нее).
- Все минтермы, которые она покрывает (101, 111), уже покрываются **a** (и 111 также покрывается **bc**).

Поэтому в минимальной ДНФ она не нужна.

Если у вас будет конкретная таблица из вашей методички с другими данными — пришлите, я помогу найти минимальное покрытие.